

PlantCare Web Application

Comprehensive Project Guide & Environment Setup Manual

1. Project Overview

PlantCare is a Django-based agricultural diagnostic and weather advisory web application. It features live leaf diagnostic scans (integrated with the Pl@ntNet API), crop-disease reference catalogs, weather advisory reports, growth suitability checkers, multi-language support (English, Hindi, Gujarati), and light/dark theme preference systems.

2. System Architecture & Modules

The project uses Django's MVT (Model-View-Template) architecture, running a Python/Django backend with a SQLite database. Front-end elements are powered by HTML5, Vanilla CSS (with custom CSS variables for light/dark theme), and Bootstrap 5 for responsiveness.

A. Accounts Module (accounts/)

Handles user accounts, localization preferences (English, Hindi, Gujarati), visual theme modes (light/dark), profile image uploads, and physical farm details (name, size, location coordinates).

B. Library Module (library/)

Stores the reference catalog for Crops, Diseases (with symptoms, causes, and treatments), and Fertilizers. This data is fully localized to display instructions in English, Hindi, and Gujarati.

C. Scans Module (scans/ & plantcare/)

Manages leaf scan uploads, uses the Pl@ntNet API to recognize plant species, matches them against localized library diseases, recommends treatment guidelines, and logs scan history.

D. Weather Module (weather/)

Integrates OpenWeatherMap for local forecast details and computes crop growth suitability indexes based on temperature/humidity.

3. Local Environment Setup Guide

Follow these steps to set up and run the PlantCare web application on your local PC.

Step 1: Create a Python Virtual Environment

Open terminal/cmd at the project root and create a virtual environment:

```
python -m venv .venv
```

Step 2: Activate the Virtual Environment

On Windows (PowerShell):

```
.venv\Scripts\Activate.ps1
```

On Windows (Command Prompt):

```
.venv\Scripts\activate.bat
```

On macOS / Linux:

```
source .venv/bin/activate
```

Step 3: Install Required Packages

Install all packages listed in requirements.txt (including Django, Pillow, requests, fpdf2, etc.):

```
pip install -r requirements.txt
```

Step 4: Configure API Credentials (.env file)

Copy the sample environment file to configure your keys:

```
copy .env.example .env
```

Open your newly created .env file in any text editor and fill in your OpenWeatherMap and Pl@ntNet API keys:

- PLANTNET_API_KEY: Your Pl@ntNet API key (my.plantnet.org)
- OPENWEATHER_API_KEY: Your OpenWeatherMap API key (openweathermap.org)

Step 5: Run Database Migrations

Create SQLite database tables:

```
python manage.py migrate
```

Step 6: Seed Crop Library & Default Admin User

Populate reference data and create the default admin profile:

```
python manage.py seed_data
```

Note: This command automatically creates the default administrator profile:

- Username: krish
- Password: krish@2007

Step 7: Launch the Server

Start Django's development server:

```
python manage.py runserver
```

Open your web browser and navigate to: <http://127.0.0.1:8000/>

4. How to Obtain and Change API Keys

Each developer running this project must configure their own private API keys in the .env file. The project relies on two external services:

A. Pl@ntNet API Key (Used for plant species identification):

1. Go to <https://my.plantnet.org/> and register a free account.
2. Navigate to your dashboard page and copy the active API key under 'My API Keys'.
3. Open the .env file in the project folder and replace the PLANTNET_API_KEY value:
PLANTNET_API_KEY=your_copied_api_key

B. OpenWeatherMap API Key (Used for live weather forecasts):

1. Go to <https://openweathermap.org/> and create a free account.
2. Click on your profile username in the top right, and go to 'My API Keys'.
3. Generate a new API key, copy it, and paste it into your .env file:
OPENWEATHER_API_KEY=your_generated_api_key

After modifying the .env file, restart the development server for changes to take effect.